PetPaws Engineering Documentation

Complete Architecture, Security, Workflow, and Implementation Manual

Version: 2.0.0 • **Architecture:** Dual React SPA + Express REST API / EJS MVC • **Author:** Abhishek Jha • **Date:** October 2026

---

1. Executive Summary & Project Purpose

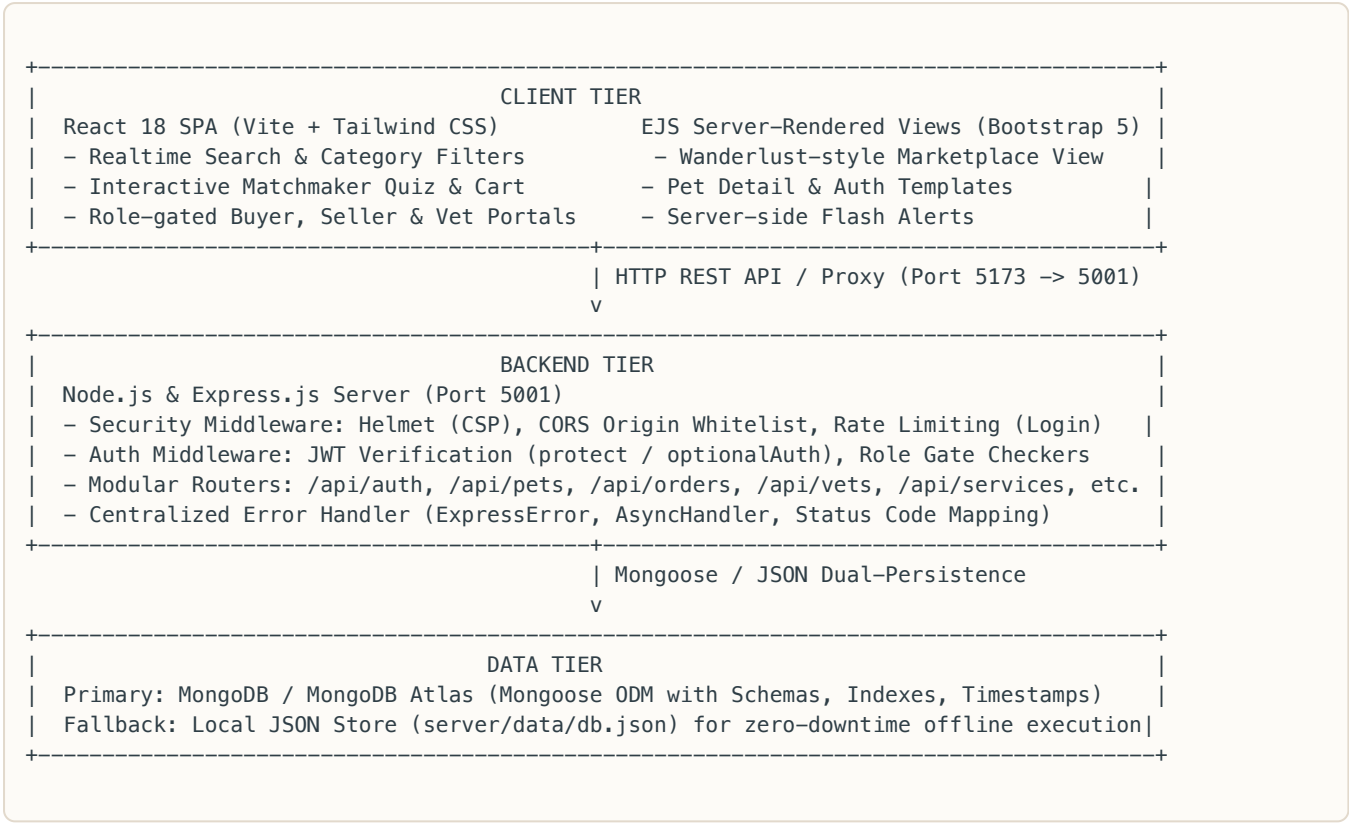
PetPaws is an integrated full-stack pet marketplace, healthcare, and adoption management platform. It bridges ethical pet breeders, certified veterinarians, specialized pet care providers, and prospective pet parents into a unified digital ecosystem.

The platform solves key pain points in traditional pet adoption and services:

- **Verified Health Assurance:** Every listing displays verified vaccination and health checks (VET VERIFIED) rather than unverified claims.
- **Dual-Client Architecture:** A modern high-performance Single-Page Application (React 18, Vite, Tailwind CSS) alongside server-rendered web views (Express, EJS, Bootstrap 5).
- **Multi-Role Role-Based Access Control (RBAC):** Distinct, strictly enforced capabilities for BUYER , SELLER , and VETERINARIAN accounts.
- **End-to-End Transaction & Lifecycle Management:** Instant checkout simulation, automated stock decrements, auto-delisting on zero inventory, 5-stage live shipment tracking, and self-service order/service cancellation with inventory restocking.
- **Telehealth & Wellness Services:** On-demand 24/7 veterinary telehealth scheduling and pet grooming/training spa reservations.

2. System Architecture & Design Patterns

PetPaws is engineered according to the classical **Model-View-Controller (MVC)** pattern with an decoupled Single Page Application frontend communicating over a JSON REST API.



Dual-Persistence Architecture: The backend dynamically checks connection status via `isMongoConnected()`. If MongoDB is connected, all operations leverage optimized Mongoose models with validation and indexes. If MongoDB is unavailable, the backend gracefully falls back to atomic file-backed reads and writes using `server/data/db.json`.

3. Technology Stack & Dependency Inventory

Layer	Package / Technology	Version	Core Responsibility
Backend Runtime	Node.js	v18+ (Tested v24)	Non-blocking asynchronous JavaScript server environment
Web Framework	Express.js	^4.22.2	REST API routing, middleware chaining, and HTTP request pipeline
Primary Database	Mongoose (MongoDB)	^8.9.5	Object Data Modeling (ODM), schemas, validation, and auto-timestamps
Authentication	jsonwebtoken	^9.0.3	Cryptographically signed stateless JWT authentication tokens
Cryptography	bcryptjs	^3.0.3	Salted password hashing (10 rounds) for user credential protection
Rate Limiting	express-rate-limit	^8.7.1	Brute-force protection on authentication endpoints (15 req/15 min)
Security Headers	helmet	^8.3.0	HTTP security headers including XSS filtering and clickjacking defense
CORS	cors	^2.8.6	Cross-Origin Resource Sharing control with configurable origin whitelist
Templating	ejs	^6.0.1	Embedded JavaScript server-side page rendering engine
Frontend Framework	React & React-DOM	^18.3.1	Declarative reactive user interface and component state management
Build Tooling	Vite	^6.4.3	Next-gen frontend bundling, hot module replacement, and API proxying
CSS Styling	Tailwind CSS	^4.3.3	Utility-first responsive styling with custom Warm Amber / Coral palette
Iconography	lucide-react	^0.469.0	Clean, accessible SVG icon components across the entire client UI

4. Project Directory & File Structure

```
PetPaws/
├── client/                          # Frontend React Application (Vite SPA)
│   ├── src/
│   │   ├── components/              # UI Modules and Interactive Sections
│   │   │   ├── Navbar.jsx           # Responsive navbar, search, user dropdown & logout
│   │   │   ├── Hero.jsx             # Welcoming hero section with verified badge
│   │   │   ├── Marketplace.jsx      # Pet grid, breed/category filters, price sorting
│   │   │   ├── PetDetailModal.jsx   # Health certificate, specs, seller info & adoption
│   │   │   ├── CheckoutModal.jsx    # Multi-method payment modal (UPI, Card, NetBanking)
│   │   │   ├── OrdersTracking.jsx   # Order timeline, carrier status & cancel option
│   │   │   ├── BuyerDashboard.jsx   # Buyer stats, order history, wishlist shortcuts
│   │   │   ├── SellerDashboard.jsx  # Inventory manager, listing uploader, payout settings
│   │   │   ├── VetSection.jsx       # 24/7 telehealth directory & appointment booking
│   │   │   ├── VetDashboard.jsx     # Veterinarian consultation manager & schedule
│   │   │   ├── GroomingCareSection.jsx  # Spa grooming packages, pricing & booking
│   │   │   ├── CustomerSupport.jsx  # FAQ accordion & collision-resistant support tickets
│   │   │   ├── PetMatchmakerQuiz.jsx  # Interactive multi-step pet compatibility quiz
│   │   │   ├── Toast.jsx            # Decoupled accessible floating toast notifications
│   │   │   ├── PawLoader.jsx        # Animated paw-print loading indicator
│   │   │   └── AuthModal.jsx        # Role-based modal (Buyer, Seller, Vet) with demo login
│   │   ├── utils/
│   │   │   └── auth.js              # JWT token persistence & authenticated fetch wrappers
│   │   ├── App.jsx                  # Root React component, active tab routing, global stat
│   │   ├── main.jsx                 # Vite entry point mounting React root
│   │   └── index.css                # Tailwind directives, animations & custom styles
│   ├── vite.config.js               # Vite configuration with Express /api proxy
│   └── package.json                 # Frontend dependencies & build scripts
├── server/                          # Backend REST API & Server-Rendered Views
│   ├── config/
│   │   └── db.js                    # Mongoose MongoDB connection & failover handler
│   ├── controllers/                 # Business Logic & Endpoint Handlers
│   │   ├── authController.js        # User registration, bcrypt login, JWT signing
│   │   ├── petController.js         # Pet CRUD, stock decrement, owner authorization
│   │   ├── orderController.js       # Checkout, buyer order isolation, status & cancel
│   │   ├── vetController.js         # Vet directory, appointments, schedule cancellation
│   │   ├── serviceController.js     # Grooming packages, reservations & cancel logic
│   │   └── supportController.js     # Collision-resistant ticket generation & inquiry CRUD
│   ├── middleware/                  # Request Pipeline Interceptors
│   │   ├── authMiddleware.js        # protect (required JWT) & optionalAuth middleware
│   │   ├── requestLogger.js         # Morgan-style formatted HTTP request logging
│   │   ├── errorHandler.js          # Centralized Express error handler & status formatting
│   │   └── asyncHandler.js          # Promise wrapper eliminating try/catch boilerplate
│   ├── models/                      # Mongoose Schemas (MongoDB)
│   │   ├── User.js                  # User credentials, roles, phone, address, payout detai
│   │   ├── Pet.js                   # Pet listing, seller ID, stock, health status, categor
│   │   ├── Order.js                 # Transaction record, tracking code, shipping, tax
│   │   ├── VetProfile.js            # Certified vet credentials, fee, specialty, schedule
│   │   ├── VetAppointment.js        # 1-on-1 consultation booking & cancellation state
│   │   ├── GroomingService.js       # Spa package offerings, pricing, durations
│   │   ├── GroomingBooking.js       # Grooming reservations & status
│   │   └── SupportTicket.js         # Customer ticket with timestamp-based unique ID
│   ├── routes/                      # Modular Express Router Declarations
│   │   ├── authRoutes.js            # /api/auth routes with login rate limiting
│   │   ├── petRoutes.js             # /api/pets routes (public GET, protected POST/PUT/DELE
│   │   ├── orderRoutes.js           # /api/orders routes (checkout, tracking, cancellation)
│   │   ├── vetRoutes.js             # /api/vets routes (directory, appointments, cancel)
```

```
│   │   ├── serviceRoutes.js              # /api/services routes (grooming, booking, cancel)
│   │   ├── supportRoutes.js              # /api/support routes (tickets, status)
│   │   └── viewRoutes.js                 # Server-rendered EJS webpage routes (/)
│   ├── views/                            # EJS Templates for Server-Side Rendering
│   ├── data/
│   │   └── db.json                       # Flat-file database fallback for offline execution
│   ├── server.js                         # Express application entry point & listener
│   └── package.json                      # Backend dependencies & scripts
├── docs/                                 # Project Documentation
│   └── PetPaws_Project_Documentation.pdf # Comprehensive compiled manual (this document)
├── package.json                          # Monorepo root scripts (dev:server, dev:client)
└── README.md                             # GitHub-ready project README
```

5. Multi-Role Feature Matrix & Workflows

5.1 Buyer Experience

- **Smart Pet Discovery:** Instant search filtering across breeds, titles, and categories (Dogs, Cats, Fishes, Rabbits, Birds) with dynamic price range filtering.
- **Health-Verified Profiles:** Interactive detail modal displaying age, gender, breed specs, seller reputation, and official health/vaccination status (`FULLY VACCINATED`).
- **Interactive Pet Matchmaker Quiz:** A multi-step lifestyle questionnaire evaluating living space, energy level, daily time commitment, and pet experience to recommend ideal pet matches.
- **Saved Wishlist:** One-click heart toggle on pet cards with persistent local storage, interactive navbar counter badge, and dedicated Saved Wishlist tab.
- **Instant Checkout:** Multi-method checkout modal supporting UPI (QR code and VPA), Credit/Debit Card, and NetBanking, including an itemized 8% tax calculation.
- **Automated Stock Management:** Purchasing a pet decrements available stock immediately. When stock reaches 0, the pet status switches to `sold` and is automatically delisted.
- **5-Stage Order Tracking:** Visual transport timeline: *Order Placed* → *Health Check* → *In Transit* → *Out for Delivery* → *Delivered* with itemized tax invoice receipt and printable slip.
- **Self-Service Order Cancellation:** Buyers can cancel any order prior to final delivery. Cancelling an order immediately refunds payment, updates the tracking status to `Cancelled`, and automatically restocks the pet listing.
- **24/7 Vet Telehealth & Spa Grooming:** Schedule vet video consultations or pet spa grooming sessions with self-service cancellation support.

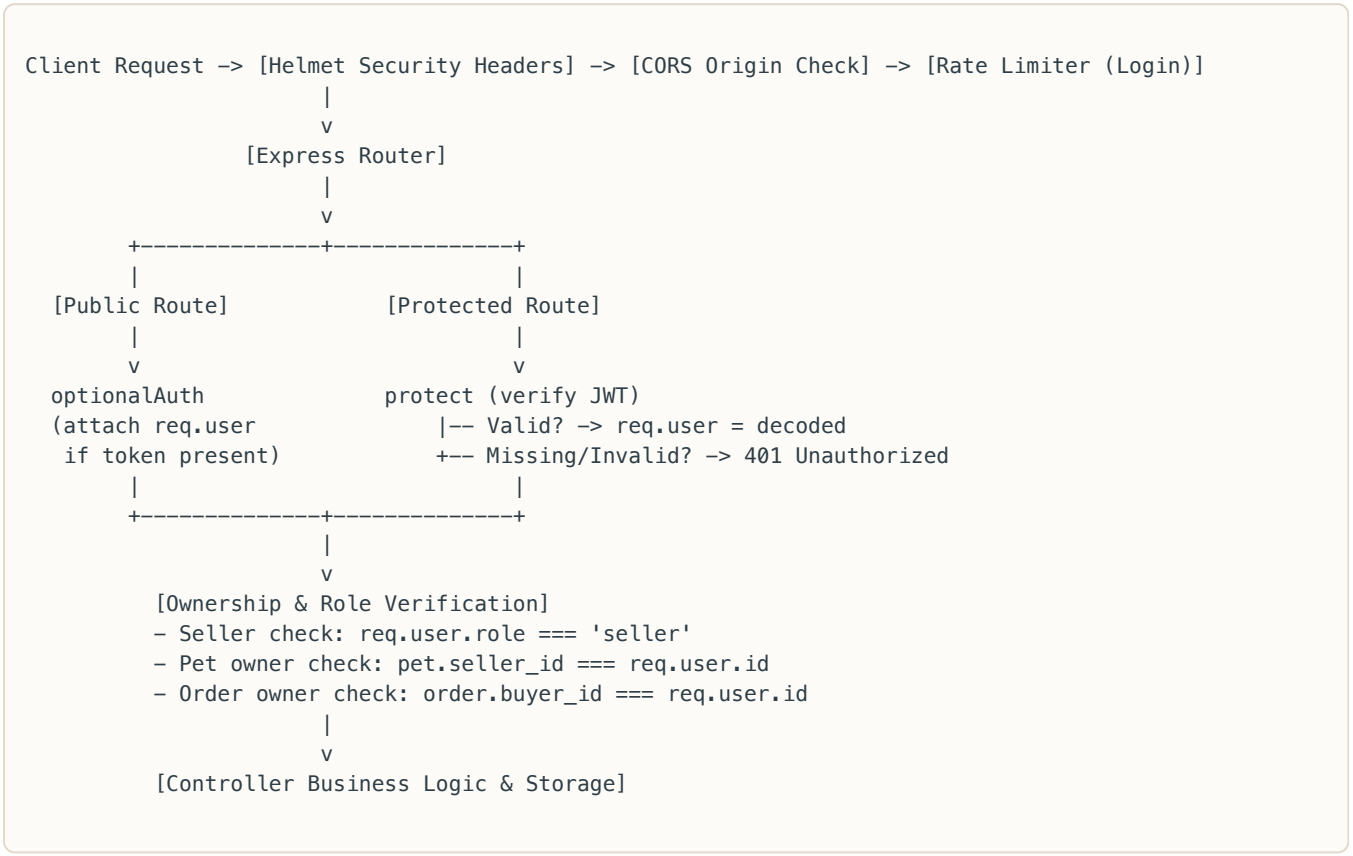
5.2 Seller Experience

- **Dedicated Seller Portal:** Strictly role-gated on both frontend and backend. Buyers and logged-out visitors are denied access.
- **Listing Lifecycle Management:** Sellers can create, edit, and delist pet inventory with custom images, health verifications, descriptions, and stock counts.
- **Strict Backend Ownership Enforcement:** Only the verified creator of a listing (matching `req.user.id` from JWT) can update or delete it. Attempts by unauthorized users are rejected with HTTP 403 Forbidden.
- **Payout Settings Configuration:** Sellers can securely configure bank accounts, IFSC codes, and UPI IDs for adoption proceeds.
- **Inventory Delisting & Restocking:** Automated stock adjustments sync in real time when orders are completed or cancelled.

5.3 Veterinarian Experience

- **Veterinary Consultation Hub:** Public profile showcase with specialties (Cardiology, Surgery, Dermatology, General Wellness), clinic hours, consultation rates, and experience metrics.
- **Vet Portal & Schedule:** Role-gated portal where veterinarians can inspect scheduled patient appointments, patient health notes, and appointment statuses.

6. Security Architecture & Role-Based Authorization



6.1 Security Measures Implemented & Verified

Security Vector	Implementation Mechanism	Enforcement Location
Password Hashing	bcryptjs salted hashing with 10 rounds	`server/controllers/authController.js` (Register)
Credential Verification	bcrypt.compare against hashed hash	`server/controllers/authController.js` (Login)
Stateless Auth	JWT signed with secret (7-day expiration)	`server/config/jwt.js` & `authMiddleware.js`
Brute-Force Defense	express-rate-limit (15 requests / 15 minutes)	`server/routes/authRoutes.js` (POST /api/auth/login)
HTTP Security Headers	helmet middleware	`server/server.js`
Listing Ownership	Enforces `pet.seller_id === req.user.id`	`server/controllers/petController.js` (PUT/DELETE)
Order Data Privacy	Queries filtered by `req.user.id`; URL tampering denied	`server/controllers/orderController.js` (GET /api/orders)
Order Cancellation Gate	Ownership check; prohibits cancelling delivered orders	`server/controllers/orderController.js` (PUT /cancel)
Mass Assignment Defense	Explicit field whitelisting on pet creation/updates	`server/controllers/petController.js`
Sensitive Field Stripping	Normalization strips `password` and `__v`	`server/utils/normalize.js`

7. Database Schema & Data Models

7.1 User Model (`server/models/User.js`)

Field	Type	Required	Description
`custom_id`	String	Yes (Unique)	Format: `u-<role>-<timestamp>`
`name`	String	Yes	Full display name of the user
`email`	String	Yes (Unique)	Normalized lowercase email address
`password`	String	Yes	Salted bcrypt password hash
`role`	String	Yes (Default: 'buyer')	Enum: `buyer`, `seller`, `vet`, `admin`
`phone`	String	No	Contact phone number
`address`	String	No	Primary delivery address
`upi_id`, `bank_account`	String	No	Seller payout configuration

7.2 Pet Listing Model (`server/models/Pet.js`)

Field	Type	Required	Description
`custom_id`	String	Yes (Unique)	Format: `pet-<id>`
`seller_id`	String	Yes	Identifier of the listing seller (from JWT)
`seller_name`	String	Yes	Display name of the seller
`title`	String	Yes	Listing title (e.g. "Golden Retriever Puppy")
`category`	String	Yes	Enum: Dogs, Cats, Fishes, Rabbits, Birds
`price`	Number	Yes	Adoption fee in USD
`stock`	Number	Yes (Default: 1)	Available count (0 marks status as 'sold')
`status`	String	Yes (Default: 'active')	Enum: `active`, `sold`, `delisted`
`health_status`	String	Yes	Vaccination / certification badge text

7.3 Order & Tracking Model (`server/models/Order.js`)

Field	Type	Required	Description
`custom_id`	String	Yes (Unique)	Format: `ORD-<6-digits>`
`buyer_id`, `seller_id`	String	Yes	User references verified via JWT
`pet_id`	String	Yes	Reference to purchased pet
`price`, `tax`, `total_amount`	Number	Yes	Financial breakdown (8% tax)
`tracking_code`	String	Yes (Unique)	Format: `TRACK-PET-<6-digits>`
`tracking_status`	String	Yes	Order Placed | Health Check | In Transit | Out for Delivery | Delivered | Cancelled
`payment_status`	String	Yes	Completed | Refunded

8. Complete REST API Specification

Method	Endpoint	Auth Required	Description
`POST`	`/api/auth/register`	No	Register new user; returns JWT token and sanitized user profile
`POST`	`/api/auth/login`	No (Rate Limited)	Authenticate credentials; returns JWT token and user profile
`PUT`	`/api/auth/update-payout`	Yes (`SELLER`)	Update bank account and UPI payout details
`GET`	`/api/pets`	Optional	Fetch listings with category, search query, and seller filters
`POST`	`/api/pets`	Yes (`SELLER`)	Create pet listing; binds `seller_id` from authenticated token
`PUT`	`/api/pets/:id`	Yes (Owner)	Update pet listing; rejects updates by non-owners
`DELETE`	`/api/pets/:id`	Yes (Owner)	Delete pet listing; rejects deletes by non-owners
`POST`	`/api/orders/checkout`	Yes (`BUYER`)	Place order; decrements stock and issues tracking number
`GET`	`/api/orders`	Yes	Get orders for authenticated buyer or seller (strict isolation)
`GET`	`/api/orders/track/:code`	Optional	Track shipment details by unique tracking code
`PUT`	`/api/orders/:id/cancel`	Yes (Buyer/Seller)	Cancel order, initiate refund, and automatically restock pet
`GET`	`/api/vets`	No	List certified veterinarians and consultation specialties
`POST`	`/api/vets/appointments`	Yes (`BUYER`)	Book 1-on-1 telehealth consultation
`PUT`	`/api/vets/appointments/:id/cancel`	Yes (Owner)	Cancel scheduled vet telehealth consultation
`GET`	`/api/services`	No	Fetch available spa, grooming, and training packages
`POST`	`/api/services/bookings`	Yes (`BUYER`)	Book grooming service package
`PUT`	`/api/services/bookings/:id/cancel`	Yes (Owner)	Cancel scheduled pet grooming reservation
`POST`	`/api/support/tickets`	No / Optional	Submit support inquiry; generates collision-resistant ID

Method	Endpoint	Auth Required	Description
`GET`	`/api/health`	No	Service health status and server timestamp check

9. Local Setup & Installation Guide (macOS)

9.1 Prerequisites

- **Node.js:** v18.0.0 or later (v20+ recommended)
- **npm:** v9.0.0 or later
- **MongoDB (Optional):** Community Edition running locally on port `27017`, or MongoDB Atlas connection string. (If MongoDB is absent, the built-in JSON fallback executes seamlessly).

9.2 Step-by-Step Installation

Step 1: Clone the repository

```
git clone <REPOSITORY_URL>
cd PetPaws
```

Step 2: Configure Environment Variables

Create `server/.env`:

```
PORT=5001
MONGODB_URI=mongodb://127.0.0.1:27017/petpaws
JWT_SECRET=petpaws-local-dev-jwt-secret-change-in-prod
JWT_EXPIRES_IN=7d
NODE_ENV=development
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:5001
```

Create `client/.env` (Optional):

```
VITE_API_URL=http://localhost:5001
```

Step 3: Install Dependencies

```
# Install backend dependencies
cd server
npm install

# Install frontend dependencies
cd ../client
npm install
cd ..
```

Step 4: Seed Initial Data (Optional)

```
cd server
npm run seed:demo
cd ..
```

Step 5: Start the Development Servers

Open two terminal windows:

Terminal 1 (Backend API Server):

```
cd server
npm start
# Output: PetPaws Server running on port http://localhost:5001
```

Terminal 2 (Frontend React Client):

```
cd client
npm run dev
# Output: Local: http://localhost:5173/
```

Visit `http://localhost:5173` in your web browser.

10. Verification, Known Limitations & Roadmap

10.1 Verified Features & Quality Audit

- ✓ **Authentication & JWT Security:** Password hashing via bcrypt, signed JWT token issuance, token storage in client localStorage, and Authorization Bearer header injection verified.
- ✓ **Listing Authorization & Ownership:** Only authenticated sellers can create listings; non-owner edit and delete requests are rejected with 403 Forbidden.
- ✓ **Order Isolation & Privacy:** Buyers can only query their own orders; URL tampering is blocked by verified JWT identity.
- ✓ **Full Lifecycle Cancellation:** Order cancellation verifies ownership, marks order `Cancelled`, sets payment status `Refunded`, and restores pet inventory to `active` stock.
- ✓ **Navbar Responsiveness:** Multi-viewport testing at 1440px, 1100px, 768px, and 375px confirms full username visibility, unclipped dedicated Logout buttons, and zero horizontal overflow.
- ✓ **Dual Persistence Engine:** Automatic failover between MongoDB Mongoose ODM and JSON flat-file storage verified.
- ✓ **Production Build Verification:** Frontend builds in 3.39s with 0 errors via Vite 6.4.3.

10.2 Known Limitations

- **Simulated Payment Gateway:** Checkout processes orders via realistic UI simulation (UPI QR, Card, NetBanking) rather than direct production webhooks (Stripe / Razorpay).
- **Simulated Transport Radar:** Carrier tracking simulation generates realistic logistics timelines rather than querying third-party GPS telemetry APIs.
- **Client Storage for Wishlist:** Wishlist items are persisted locally in client localStorage rather than synchronized across multiple devices via database record.

10.3 Future Engineering Roadmap

- **Payment Gateway Integration:** Connect production Stripe and Razorpay webhook handlers for real-world card processing and automated refund dispatches.
- **Real-Time Chat & Telehealth:** Implement Socket.io or WebRTC for live instant messaging and peer-to-peer video calls between pet parents and veterinarians.
- **Live Push Notifications:** Web Push and SMS notifications alerting pet parents upon shipment status advancements.
- **Automated Test Suite:** Integrate Jest and React Testing Library for CI/CD test automation.

11. Glossary & Developer Learning Guide

Concept / Term	Definition & Role in PetPaws
JWT (JSON Web Token)	A compact, URL-safe token containing user ID and role, cryptographically signed by the backend to verify identity statelessly on every protected request.
Bcrypt Salt Hashing	A cryptographic technique adding random bytes to passwords before hashing to make rainbow table dictionary attacks infeasible.
MVC Pattern	Model-View-Controller architecture separating data schemas (Models), presentation (Views/React), and route logic (Controllers).
Dual Persistence	An enterprise resiliency pattern where the app seamlessly functions against MongoDB or automatically degrades to a local JSON database if MongoDB is down.
Vite Proxy	A development server configuration forwarding client `/api/*` HTTP requests to the Express server on port 5001, avoiding CORS restrictions.
Atomic Stock Decrement	Decreasing pet inventory at the moment of checkout and flipping status to 'sold' when stock reaches 0 to prevent overselling.